NAQLA

LAB 3

Lab 3 — Ship It Async

Agentic Backend Engineering

Agentic Backend Engineering — Build Scalable Backends With and For AI Agents



Lab 3 — Ship It Async

Overview

Goal

Shahna's API works for one customer clicking slowly. Today you make it hold up under load: no N+1 queries, slow work moved to queues, no double bookings when a phone retries a request, cached reference data, and rate limits. You will **measure before and after** with k6, and you will not believe any performance claim (yours or the agent's) that is not backed by a number.

Today's habit: **the agent proposes, the numbers decide.** When the agent says "this will be faster", your reply is "show me".

What you will build

Piece	Where it lives
k6 load script and before/after table	`load/booking.js`, `docs/plans/day3-perf.md`
N+1 fix on `GET /shipments` with a query-count test	`ShipmentController`, `tests/Feature/PerformanceTest.php`
Horizon: `supervisor-1` for `dispatch`, `notifications`, `default`; `supervisor-ai` for `ai`	`config/horizon.php`
`ShipmentBooked` event and its `QueueBookingJobs` listener	`app/Events/`, `app/Listeners/`
`AssignVehicleJob` (row locks) and `SendBookingConfirmationJob`	`app/Jobs/`
`POST /api/dispatch/shipments/{reference}/assign`	staff-only manual override, `BookingService::assign()`
`EnsureIdempotency` middleware (alias `idempotent`) and `IdempotencyKey` model	`app/Http/Middleware/`, `app/Models/`
Cached cities and distances	`CityController`, `PricingService`
Rate limiter `api` (60/min) and `AssignRequestId` middleware	`app/Providers/AppServiceProvider.php`, `bootstrap/app.php`

Starting point

```
git checkout day2-end
git switch -c my-day3
sail up -d
sail artisan migrate:fresh --seed
sail artisan test
```

```
  Tests:    56 passed
```

Install k6 on your laptop (`brew install k6` on macOS, `winget install k6` on Windows), or use Docker as shown in Step 1.

Time budget

Part	Step	Minutes
Part 1 (11:00–12:30)	1. Load data and a k6 baseline	20
	2. Find the N+1, by hand	15
	3. Fix the N+1 test-first	15
	4. Install Horizon	10
	5. `ShipmentBooked`, jobs and locking	25
	Checkpoint 1	5
Part 2 (14:15–15:45)	6. Dispatcher manual assign	15
	7. Idempotency keys	25
	8. Cache cities and distances	10
	9. Rate limits and request IDs	15
	10. k6 after, by hand	15
	Checkpoint 2 and commit	10

WARNING

Sail serves the app with `php artisan serve`, which is not a production web server. Your k6 numbers are only meaningful **compared with each other on the same laptop**. Never quote them as “Shahna handles N requests per second”.

Part 1 — Measure, Fix, Go Async (11:00–12:30)

Step 1 — Load data and a k6 baseline (20 min)

What and why. You cannot improve what you have not measured. Before changing anything, you need a repeatable load script that exercises the booking flow: quote, book, list.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Small diffs.

Create load/booking.js for k6:
- BASE_URL from __ENV.BASE_URL, default http://localhost;
  number of VUs from __ENV.VUS, default 20
- setup(): GET /api/cities and return the city ids
- default function: on its FIRST iteration each VU registers its own
  account via POST /api/auth/register (unique email using uuidv4 from
  https://jslib.k6.io/k6-utils/1.4.0/index.js) and keeps the token.
  Every iteration: POST /api/quotes (two random cities, random
  vehicle_type, weight_kg 100-1100), POST /api/shipments with that
  quote_id and pickup_at tomorrow, then GET /api/shipments.
  Tag the requests {name: 'quote'}, {name: 'book'}, {name: 'list'}.
  check() status codes (201, 201, 200). sleep(3) at the end.
- Send an Idempotency-Key header (uuidv4) on POST /api/shipments
  (unused today until Step 7, harmless now).
- options: stages ramp up 20s, hold 1m, ramp down 10s; thresholds
  http_req_failed rate<0.01, p(95)<300 for {name:list},
  p(95)<500 for {name:book}.
Do not change any application code.
```

What to check.

- Good: `sleep(3)` per iteration. Each user then makes 3 requests every 3+ seconds, which stays under the 60/min rate limit you add in Step 9. Without that, your "after" run would be full of 429s.
- Good: thresholds are per tag, for example `'http_req_duration{name:list}': ['p(95)<300']`. k6 only prints per-tag numbers for metrics that have thresholds.
- Red flag: the script registers a new user every iteration. That measures bcrypt, not your API. One account per VU is enough.
- Red flag: the agent invents k6 APIs. `http`, `check`, `sleep`, `group` come from `k6/http` and `k6`; there is no `k6.request()`.

Run the baseline (the database was seeded in the starting point):

```
k6 run load/booking.js
```

Without a local k6, use Docker (`host.docker.internal` reaches Sail from inside the container):

```
docker run --rm -i -e BASE_URL=http://host.docker.internal grafana/k6 run - < load/booking.js
```

The end of the report looks like this (your numbers will differ):

```
  × http_req_duration{name:book}...: avg=160ms  p(95)=520ms
  × http_req_duration{name:list}...: avg=240ms  p(95)=410ms
    http_req_failed................: 0.00%
    http_reqs......................: 1240   12.4/s
```

Create `docs/plans/day3-perf.md` **by hand** and copy in the two p95 values, `http_req_failed` and the total requests per second, under a heading “Baseline”. Commit:

```
git add load docs/plans/day3-perf.md
git commit -m "chore: k6 baseline"
```

Step 2 — Find the N+1, by hand (15 min)

What and why. `GET /api/shipments` is your slowest endpoint. You will find out why **without the agent**, because “count the queries” is a skill you need when the agent is not there.

> **WARNING**
>
> **By hand.** Do not ask the agent to find the problem. Find it, then let the agent fix it.

Open Tinker and count queries for one request’s worth of work:

```
sail artisan tinker
```

```
> DB::enableQueryLog();
> $u = App\Models\User::where('email', 'customer@shahna.test')->first();
> App\Http\Resources\ShipmentResource::collection($u->shipments()->latest()->paginate(15))
      ->resolve();
> count(DB::getQueryLog());
= 32
> collect(DB::getQueryLog())->pluck('query')->take(6);
```

You will see the same `select * from "cities" where "cities"."id" = ? limit 1` repeated again and again: one query per shipment for `origin` and one for `destination`. With the pagination count and the page itself that is **2 + 2 × 15 = 32** queries for one page. The seeded demo shipments have no vehicle yet; once vehicles are assigned, `vehicle` adds a third query per shipment. That is the N+1 problem.

A second way to see it is to make Laravel refuse lazy loading in development. In `AppServiceProvider::boot()` add:

```
use Illuminate\Database\Eloquent\Model;

Model::preventLazyLoading(! app()->isProduction());
```

Now call the endpoint:

```
curl -s http://localhost/api/shipments \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" | jq .message
```

```
"Attempted to lazy load [origin] on model [App\\Models\\Shipment] but lazy loading is
        disabled."
```

Remove the `preventLazyLoading` line again once you have seen the error. It is a good diagnostic, but other code paths load relations lazily on purpose (for example `SendBookingConfirmationJob` reads `$shipment->user`). The reference solution guards `GET /api/shipments` with a query-count test instead (Step 3), so a new N+1 there breaks a test instead of slowing down production.

Step 3 — Fix the N+1 test-first (15 min)

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Test first.

1. In tests/Feature/PerformanceTest.php add a test that counts the
   queries of GET /api/shipments (DB::enableQueryLog() /
   count(DB::getQueryLog())) for a user with 2 shipments, then again
   after adding 10 more, and asserts both counts are equal. Run it; it
   must fail.
2. Fix ShipmentController@index with eager loading: with(['origin',
   'destination', 'vehicle']). Do not change ShipmentResource's output
   shape.
3. The list filters by user_id and orders by created_at. Check that the
   Day 1 shipments migration already has an index on (user_id,
   created_at); do not add a second one.
Run the full test suite and report the query count before and after.
```

What to check.

- Good: `ShipmentResource` uses `$this->whenLoaded('vehicle')` or the controller always eager-loads, so the resource never triggers a query on its own.
- Good: the test compares two counts (2 shipments vs 12) instead of a magic number, so it fails as soon as the count grows with the page size.
- Red flag: the fix caches the whole list in Redis. That hides the N+1 instead of fixing it, and serves stale data after a cancel.

- Red flag: the query-count assertion was loosened to make it pass (for example, `toBeLessThan(50)`).

Check the plan of the real query in PostgreSQL by hand (the seeded customer is user 1):

```
sail psql -c "EXPLAIN SELECT * FROM shipments WHERE user_id = 1 ORDER BY created_at DESC LIMIT
        15;"
```

```
 Limit  (cost=0.28..12.45 rows=15 width=...)
   ->  Index Scan Backward using shipments_user_id_created_at_index on shipments ...
```

`Index Scan` is what you want. `Seq Scan` on a large table means the index is not used.

Step 4 — Install Horizon (10 min)

What and why. Horizon runs and monitors Redis queue workers. You will use four queues, and put `ai` on its own supervisor, so that slow AI work (Day 4) can never block vehicle assignment.

The `laravel/horizon` package is already in `composer.json` since `day1-start`. Publish its config and service provider:

```
sail artisan horizon:install
```

In `.env` check `QUEUE_CONNECTION=redis`. Then edit the `defaults` in `config/horizon.php` **by hand**: `supervisor-1` processes three queues in priority order, and a second supervisor handles only `ai`:

```
'defaults' => [
    'supervisor-1' => [
        'connection' => 'redis',
        'queue' => ['dispatch', 'notifications', 'default'],
        'balance' => 'auto',
        // ...keep the other generated keys...
        'tries' => 3,
        'timeout' => 60,
    ],
    // LLM calls are slow and expensive: give them their own workers and a longer timeout.
    'supervisor-ai' => [
        'connection' => 'redis',
        'queue' => ['ai'],
        'balance' => 'simple',
        'maxProcesses' => 1,
        'memory' => 128,
        'tries' => 2,
        'timeout' => 180,
    ],
],
```

Leave `environments` as generated: it only sets `maxProcesses` (10 in `production`, 3 in `local`) for `supervisor-1`.

Start Horizon in a **second terminal** and leave it running:

```
sail artisan horizon
```

```
  INFO  Horizon started successfully.
```

Open `http://localhost/horizon`. The dashboard is open in the `local` environment only; in production, access is controlled by the `viewHorizon` gate in `app/Providers/HorizonServiceProvider.php` (the reference solution allows admins only). The reference also schedules `horizon:snapshot` every five minutes in `routes/console.php` so the metrics graphs fill in.

> **TIP**
> Horizon does not reload code. After you change a job class, stop Horizon with `Ctrl+C` and start it again, or you will debug yesterday's code.

Step 5 — `ShipmentBooked`, jobs and locking (25 min)

What and why. Booking currently does everything inside the HTTP request. Assigning a vehicle and sending a confirmation are slow and can fail, so they move to queues. The request only books the shipment and fires `ShipmentBooked`.

Three rules matter here:

1. **Dispatch after commit.** `BookingService::book()` fires `ShipmentBooked` *inside* its `DB::transaction`. If a worker picks up the job before that transaction commits, it cannot see the shipment yet (or sees a half-saved one). Making the event implement `ShouldDispatchAfterCommit` holds it back until the commit, and drops it if the transaction rolls back.
2. **Re-fetch under a lock.** A queued job carries a serialized copy of the shipment. By the time it runs, a customer may have cancelled it or a dispatcher may have assigned it. So the job opens a transaction and re-reads the row with `Shipment::lockForUpdate()->find(...)` before it decides anything. `BookingService::cancel()` already does the same since Day 2.
3. **Lock the vehicle row.** Two workers assigning two shipments at the same moment could both pick the same free van. `lockForUpdate()` on the vehicle query, inside the transaction, makes the second worker wait until the first commits.

Booking already moves the shipment `pending → confirmed` (two `ShipmentEvent`s, and the API returns `"status": "confirmed"`). The status machine allows `confirmed → assigned`, so the job only assigns, and only if the re-read shipment is still `confirmed`. If no vehicle is free, the shipment stays `confirmed` for a dispatcher.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first. Small diffs.

1. App\Events\ShipmentBooked (carries the Shipment), implementing
   ShouldDispatchAfterCommit. Fire it from BookingService::book(), after
   the shipment is confirmed, inside the existing transaction.
2. Listener App\Listeners\QueueBookingJobs that dispatches:
   - AssignVehicleJob on the "dispatch" queue
   - SendBookingConfirmationJob on the "notifications" queue
3. AssignVehicleJob(Shipment):
   - in DB::transaction: re-read the shipment with
     Shipment::lockForUpdate()->find(id); if its status is not
     confirmed (cancelled, or already assigned), return. The job must be
     safe to run twice.
   - find the smallest available Vehicle with type = shipment
     vehicle_type, capacity_kg ⩾ weight_kg, current_city_id = origin,
     orderBy('capacity_kg'), using lockForUpdate()
   - if found: vehicle status busy, shipment vehicle_id set, transition
     confirmed → assigned with transitionTo() (ShipmentEvent
     "Vehicle <plate> assigned.")
   - if none: Log::warning with the reference and leave status confirmed
   - add a migration with an index on vehicles (current_city_id, type,
     status) to match that WHERE clause
4. SendBookingConfirmationJob: Mail::raw plain-text confirmation to the
   user (reference, pickup time, price); MAIL_MAILER=log in development.
Tests:
- booking dispatches ShipmentBooked (Event::fake)
- booking pushes both jobs on the right queues (Queue::fake,
  Queue::assertPushedOn)
- AssignVehicleJob picks the smallest vehicle that fits in the origin
  city, skipping busy, wrong type, too small and wrong-city vehicles
- no vehicle → status confirmed, vehicle_id null
- shipment cancelled before the job runs → job does nothing
Existing booking tests stay unchanged: the response status is still
"confirmed".
```

What to check.

- Good: `lockForUpdate()` is called **inside** `DB::transaction`. Outside a transaction, the lock is released immediately and does nothing.
- Good: the job re-reads the shipment with `lockForUpdate()` instead of trusting `$this->shipment->status`. Queued jobs serialize the model's id and re-fetch it when the job starts, but without a lock the status can still change between that read and the update (for example, a cancel arriving at the same moment).
- Good: `ShipmentBooked implements ShouldDispatchAfterCommit`. Without it the tests still pass (they run in one transaction with a fake queue), so read the event class yourself.
- Red flag: `Vehicle::where(...)->get()->first()` then `->lockForUpdate()`. The lock must be part of the query that selects the row.

- Red flag: the job calls `sleep()` or retries forever. Use Horizon's `tries` and the job's `$backoff`.
- Red flag: `pending → assigned` in one step, or status changes that bypass `canTransitionTo`.

Prove the lock works **by hand**. In Tinker, create a scenario with two shipments and only one matching vehicle, then dispatch both jobs while Horizon (3 processes) is running:

```
> $v = App\Models\Vehicle::where('status','available')->first();
> App\Models\Vehicle::where('id','<>',$v->id)->where('type',$v->type)
      ->where('current_city_id',$v->current_city_id)->update(['status'=>'maintenance']);
> $a = App\Models\Shipment::factory()->create(['vehicle_type'=>$v->type,'origin_city_id'=>$v
      ->current_city_id,'weight_kg'=>100]);
> $b = App\Models\Shipment::factory()->create(['vehicle_type'=>$v->type,'origin_city_id'=>$v
      ->current_city_id,'weight_kg'=>100]);
> App\Jobs\AssignVehicleJob::dispatch($a)->onQueue('dispatch');
      App\Jobs\AssignVehicleJob::dispatch($b)->onQueue('dispatch');
> [$a->fresh()->status, $b->fresh()->status];
```

```
= [App\Enums\ShipmentStatus::Assigned, App\Enums\ShipmentStatus::Confirmed]
```

Exactly one shipment gets the vehicle. Check the Horizon dashboard: both jobs are under **Completed**, none under **Failed**.

CHECKPOINT

Checkpoint 1 (12:30)

```
sail artisan test
```

```
  Tests:    60+ passed
```

Zero failures, and more tests than the 56 you started with. Then book a shipment by curl (with Horizon running) and check its tracking a few seconds later:

```
curl -s http://localhost/api/shipments/$REF/tracking \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" | jq
        '[.data[].status]'
```

```
["pending", "confirmed", "assigned"]
```

`["pending", "confirmed"]` is also correct if no vehicle of that type is free in the origin city.

```
git add -A
git commit -m "feat: async booking with vehicle assignment"
```

Part 2 — Idempotency, Caching, Limits (14:15–15:45)

Step 6 — Dispatcher manual assign (15 min)

What and why. When the automatic job finds no vehicle, a dispatcher assigns one by hand. This is the same locking problem as the job, so the assignment logic should live in **one** place that both call.

🤖 AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first.

POST /api/dispatch/shipments/{shipment:reference}/assign, body {vehicle_id}.
- ShipmentPolicy::assign: staff only (dispatcher or admin, User::isStaff()).
- FormRequest AssignVehicleRequest: vehicle_id required, integer,
  exists:vehicles,id.
- Move the "assign this vehicle to this shipment" logic out of
  AssignVehicleJob into BookingService::assign(Shipment, Vehicle, note),
  used by both the job and this endpoint. In its own DB::transaction,
  re-read BOTH rows with lockForUpdate() and re-check them.
- 422 with errors.vehicle_id if the vehicle is not available, wrong type,
  or too small; 422 with errors.status if the shipment is not confirmed
  or assigned. Re-assigning an assigned shipment frees the old vehicle.
- Return 200 with ShipmentResource (vehicle included).
Tests: dispatcher success; re-assign frees the old vehicle; customer →
403; busy or unsuitable vehicle → 422; cancelled shipment → 422.
Existing job tests must still pass unchanged.
```

What to check.

- Good: `git diff tests/` shows only new tests; the Step 5 job tests are untouched and green.
- Red flag: the endpoint and the job each have their own copy of the vehicle checks.
- Red flag: the route is not behind `auth:sanctum`, or there is no `Gate::authorize('assign', $shipment)` at all.
- Note: the FormRequest validates `vehicle_id` before the controller's policy check runs, so a customer who sends an unknown id gets 422 rather than 403. The reference solution accepts that; adding an `authorize()` method to `AssignVehicleRequest` would close it.

Step 7 — Idempotency keys (25 min)

What and why. A phone on a weak 3G connection in Sabha sends `POST /api/shipments`, the response gets lost, and the app retries. Without protection, the customer may be charged for two trucks. The unique `quote_id` saves you for this endpoint, but the client gets a confusing 422. The industry pattern is an **Idempotency-Key** header: the client sends a unique key per logical action; the server stores the first response and replays it for retries.

The rules for Shahna:

Situation	Response
No `Idempotency-Key` header	400 `{"message": "The Idempotency-Key header is required."}`
New key	Run the request, store status + body, return it
Same key, same user, same body	Replay the stored status and body, header `Idempotent-Replayed: true`
Same key, same user, different body	422 `{"message": "This Idempotency-Key was already used for a different request."}`
Same key, first request still running	409 `{"message": "A request with this Idempotency-Key is still being processed."}`
Same key, first request started more than 60 seconds ago and never finished	Treat as abandoned (the worker crashed): run the request again
First request ended in a 5xx	The key is deleted, so the client can retry with the same key

Keys are scoped per user and expire after one day.

The 60-second rule matters. Without it, a PHP process that dies between “claim the key” and “store the response” leaves a row with no response forever, and every retry of that booking gets 409 until the key expires a day later.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first, small diffs.

1. Migration + model IdempotencyKey: key (string), user_id (FK),
   request_hash (char 64), response_status (nullable small int),
   response_body (nullable text), expires_at (indexed). Unique index on
   (user_id, key). Make the model MassPrunable for rows past expires_at;
   model:prune is scheduled daily in routes/console.php.
2. Middleware App\Http\Middleware\EnsureIdempotency, alias "idempotent"
   registered in bootstrap/app.php, applied ONLY to POST /api/shipments.
   - missing header, or longer than 255 chars → 400
   - request_hash = sha256 of method + path + raw body
   - claim the key BEFORE running the request with
     IdempotencyKey::createOrFirst(['user_id', 'key'], [...]) (it relies
     on the unique index), then store status + body afterwards
   - same key + different hash → 422; same key + no response yet → 409;
     same key + stored response → replay it with header
     Idempotent-Replayed: true
   - a key that is expired, or has no response and was last updated more
     than IN_FLIGHT_TIMEOUT_SECONDS = 60 ago, is deleted and claimed again
   - on a 5xx response delete the row so the client can retry
3. Update load/booking.js only if needed (it already sends the header).
Tests: each row of the table, including the abandoned-request row (use
$this->travel()); keys are per user (user B can use user A's key);
replay returns the same body and no second Shipment exists.
```

What to check.

- Good: the row is **inserted before** the controller runs. “Check if it exists, then insert later” has a race: two retries arrive together, both see nothing, both book.
- Good: a test creates a key with no response, travels two minutes back, and checks that the next request runs (201) instead of getting 409.
- Good: the replay response is byte-for-byte the first response (same `reference`).
- Red flag: the key is global instead of per user. User B could replay user A’s response and see their shipment.
- Red flag: the middleware applies to every route, including `GET`. GETs are already idempotent.
- Red flag: the stored body is cached in Redis with no database row. If Redis is flushed, the double-booking protection silently disappears.

Test it by hand with a retry, then with two parallel requests:

```
KEY=$(uuidgen)
Q=$(curl -s -X POST http://localhost/api/quotes -H "Accept: application/json" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"origin_city_id":1,"destination_city_id":3,"weight_kg":500,"vehicle_type":"van"}' | jq
      .data.id)

for i in 1 2; do
  curl -s -i -X POST http://localhost/api/shipments \
    -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
    -H "Content-Type: application/json" -H "Idempotency-Key: $KEY" \
    -d "{\"quote_id\":$Q,\"pickup_at\":\"2026-10-26T08:00:00Z\"}" | grep -E "HTTP/|Idempotent-
      Replayed|reference"
done
```

```
HTTP/1.1 201 Created
{"data":{"reference":"SHN-P4M9QX",...
HTTP/1.1 201 Created
Idempotent-Replayed: true
{"data":{"reference":"SHN-P4M9QX",...
```

Send the same key once more with a different `pickup_at`: you get 422 and `"This Idempotency-Key was already used for a different request."`

Same reference twice, one shipment in the database. Now without the header:

```
{"message":"The Idempotency-Key header is required."}
```

Step 8 — Cache cities and distances (10 min)

What and why. Cities and distances almost never change but are read on every quote. They are perfect cache candidates. The hard part of caching is not storing, it is **invalidating**.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first.

1. GET /api/cities: Cache::remember(City::CACHE_KEY, now()->addDay(), ...)
   with City::CACHE_KEY = 'cities'.
2. PricingService::distanceKm(): cache the km lookup per pair with key
   "distance:{from}:{to}" for one week (road distances do not change).
3. Invalidation: in City::booted(), forget City::CACHE_KEY when a City
   is saved or deleted.
Tests: the second GET /api/cities runs zero queries (Cache uses the
array store in tests); updating a city's name_en changes the next
/api/cities response.
```

What to check.

- Good: `.env` has `CACHE_STORE=redis`; `phpunit.xml` keeps `CACHE_STORE=array` so tests do not touch Redis.
- Red flag: caching `GET /api/shipments` or anything per user without the user id in the key. User A sees user B's shipments.
- Red flag: no invalidation for cities, "because cities never change". The test with a renamed city must exist. (Distances are the one deliberate exception: if you ever change them, run `sail artisan cache:clear`.)

Check Redis by hand:

```
sail redis redis-cli --scan --pattern '*cities*'
```

```
shahna-database-shahna-cache-cities
```

(The prefix depends on your `APP_NAME` and `CACHE_PREFIX`.)

Step 9 — Rate limits and request IDs (15 min)

What and why. Rate limiting protects you from a buggy client looping, and later from someone running up your AI bill. Request IDs let you follow one request through logs and queued jobs.

AGENT PROMPT

Agent prompt

```
Plan first, then wait for my approval. Tests first.

1. In AppServiceProvider::boot define:
   RateLimiter::for('api', 60 per minute by user id, falling back to IP)
   Apply it to all API routes via $middleware->throttleApi() in
   bootstrap/app.php. (A tighter "agent" limiter comes on Day 5.)
2. Middleware App\Http\Middleware\AssignRequestId, prepended to the
   global stack in bootstrap/app.php: read X-Request-Id or generate a
   UUID, add it to every log line with
   Log::shareContext(['request_id' => ...]), and return it as the
   X-Request-Id response header.
Tests: the 61st request to GET /api/shipments in a minute returns 429;
responses include X-Request-Id; an incoming X-Request-Id is echoed.
```

What to check.

- Good: `Limit::perMinute(60)->by($request->user()?->id ?: $request->ip())`.
- Good: the 429 test calls `RateLimiter::clear(...)` or uses fresh users so it does not leak into other tests.
- Red flag: the limiter is keyed only by IP. Everyone in the training room shares one IP behind the router.

```
for i in $(seq 1 62); do
  curl -s -o /dev/null -w "%{http_code} " http://localhost/api/shipments \
    -H "Accept: application/json" -H "Authorization: Bearer $TOKEN"
done; echo
```

```
200 200 200 ... 200 429 429
```

Wait a minute for the limiter to reset, then:

```
curl -s -i http://localhost/api/shipments -H "Accept: application/json" \
  -H "Authorization: Bearer $TOKEN" | grep -iE "x-ratelimit|x-request-id"
```

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
X-Request-Id: 9b2f0c1e-6a7d-4f0e-9b9e-2a1c3d4e5f60
```

Step 10 — k6 after, by hand (15 min)

> **WARNING**
>
> **By hand.** Run k6 and fill in the table yourself. Then ask the agent to explain the numbers, and check its explanation against what you changed.

Wait one minute for the rate limiter to reset, then:

```
sail artisan migrate:fresh --seed
k6 run load/booking.js
```

Add an "After" section to `docs/plans/day3-perf.md`:

Metric	Baseline	After	Change
p95 `list`			
p95 `book`			
Queries per `GET /api/shipments` (Step 2)	32		
`http_req_failed`			
Requests per second			

Typical result: `list` p95 drops because the page now runs a fixed number of queries; `book` gets faster because assignment and the email moved to queues; `http_req_failed` stays at 0% because each user stays under 60 requests per minute.

AGENT PROMPT

Agent prompt

```
Read docs/plans/day3-perf.md and `git log day2-end..HEAD --oneline`.
For each row of the table, explain which commit most likely caused the
change. If a number did not improve or got worse, say so and propose one
experiment to find out why. Do not change code. Mark every claim you
cannot prove from the numbers as a hypothesis.
```

If the agent claims an improvement your table does not show, it is wrong. Write that down for the reflection.

CHECKPOINT

Checkpoint 2 (15:45)

```
sail artisan test
```

```
  Tests:    75+ passed
```

The `day3-end` reference has 80 tests; your count will be close to that.

```
# Missing idempotency key
curl -s -X POST http://localhost/api/shipments -H "Accept: application/json" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{}'
```

```
{"message":"The Idempotency-Key header is required."}
```

```
# Customer cannot use the dispatcher route
curl -s -X POST http://localhost/api/dispatch/shipments/$REF/assign \
  -H "Accept: application/json" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"vehicle_id":1}'
```

```
{"message":"This action is unauthorized."}
```

Horizon dashboard: zero failed jobs. `docs/plans/day3-perf.md` has both tables filled in.

```
git add -A
git commit -m "feat: idempotency, caching, rate limits, perf report"
```

If you're stuck

The reference solution is the `day3-end` tag.

```
git diff --stat day3-end
git show day3-end:app/Jobs/AssignVehicleJob.php
git diff day3-end -- bootstrap/app.php
git show day3-end:load/booking.js > /tmp/booking-ref.js && k6 run /tmp/booking-ref.js
```

Common problems:

Symptom	Likely cause
Jobs stay in "Pending" in Horizon	Horizon not running, or `QUEUE_CONNECTION` still `database` / `sync`
Job fails with "No query results for model"	Event dispatched before the transaction committed
Your code change has no effect on jobs	Horizon not restarted
k6 shows many 429s	Missing `sleep(3)`, or all VUs share one token
Every retry of a booking gets 409	A request crashed after claiming the key; it is reclaimed after 60 seconds
k6 in Docker: connection refused	Use `BASE_URL=http://host.docker.internal`

To restart from the reference:

```
git stash
git switch -c my-day3-reset day3-end
sail composer install
sail artisan migrate:fresh --seed
sail artisan test
```

Stretch goals

1. **SKIP LOCKED.** Change the vehicle query to `FOR UPDATE SKIP LOCKED` (`->lock('for update skip locked')`) so a second worker picks the *next* free vehicle instead of waiting. Explain when that is better and when it is worse.
2. **Pulse.** Install Laravel Pulse and find your slowest query and slowest job in its dashboard during a k6 run.
3. **Spike test.** Add a second k6 scenario using the `ramping-vus` executor that goes from 0 to 100 VUs in 30 seconds. Where does it break first: PHP, PostgreSQL connections, or Redis?
4. **Failed job handling.** Add a `failed()` method to `AssignVehicleJob` that records a `ShipmentEvent` note "Automatic assignment failed" and logs the shipment reference.

5. **Request IDs in jobs.** `Log::shareContext()` only covers the HTTP request. Switch `AssignRequestId` to Laravel's `Context` facade so the request ID also appears in the logs of the jobs that request queued.

Reflection

1. Which performance claim did the agent make today that your k6 numbers did **not** support? If none, which claim did you check, and how?
2. Explain in your own words why the idempotency row must be inserted *before* the controller runs. What test would catch a "check then insert" implementation, and did you write it?
3. You locked the vehicle row in `AssignVehicleJob`. Describe the exact double-assignment that happens without the lock, using two workers and one van in Benghazi.